Vivekanand Education Society's Institute Of Technology
Department Of Information Technology
DSA mini Project
A.Y. 2024-25

Title: Single Player Snake Game.

Sustainability Goal: "Quality Education"

Domain: Game Development

Mentor Name: Mrs. Kajal Jewani.

Group Members:
Member1 :Aarya Thorat(61)
Member2: Nikhil Sunchu (59)

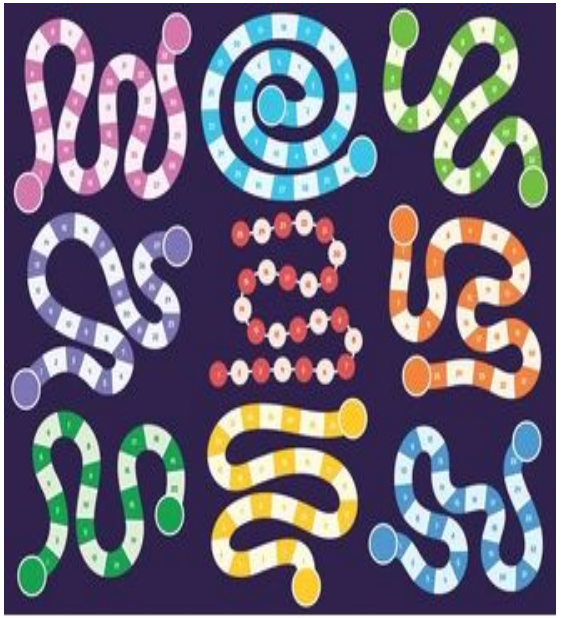

1 NO POVERTY
2 ZERO HUNGER
3 GOOD HEALTH AND WELL-BEING
4 QUALITY EDUCATION
5 GENDER EQUALITY
6 CLEAN WATER AND SANITATION
7 AFFORDABLE AND CLEAN ENERGY
8 DECENT WORK AND ECONOMIC GROWTH
9 INDUSTRY, INNOVATION AND INFRASTRUCTURE
10 REDUCED INEQUALITIES
11 SUSTAINABLE CITIES AND COMMUNITIES
THE GLOBAL GOALS
For Sustainable Development
12 RESPONSIBLE CONSUMPTION AND PRODUCTION
13 CLIMATE ACTION
14 LIFE BELOW WATER
15 LIFE ON LAND
16 PEACE AND JUSTICE STRONG INSTITUTIONS
17 PARTNERSHIPS FOR THE GOALS

Content

1. Introduction to the Project
2. Problem Statement
3. Objectives of the Project
4. Scope of the Project
5. Requirements of the System (Hardware, Software)
6. ER Diagram of the Proposed System
7. Data Structure & Concepts Used
8. Algorithm Explanation
9. Time and Space Complexity
10. Front End
11. Implementation
12. Gantt Chart
13. Test Cases
14. Challenges and Solutions
15. Future Scope
16. Code
17. Output Screenshots
18. Conclusion
19. References (in IEEE Format)

Introduction to Project

- Revitalizes the classic single-player Snake game using C
- Integrates essential Data Structures and Algorithms (DSA) concepts
- Offers an entertaining experience with a learning focus
- Enhances programming skills through hands-on practice
- Emphasizes user interaction, algorithm efficiency, and resource management



Problem Statement

Despite the enduring popularity of Snake games, many versions fail to incorporate structured programming principles, leading to a lack of educational value. Additionally, players often encounter limited engagement due to repetitive gameplay mechanics and poor resource management. This project addresses these issues by developing a single-player Snake game that utilizes fundamental DSA concepts, such as linked lists and arrays, to create a more dynamic and instructive gaming experience. The objective is to combine entertainment with education, fostering a deeper understanding of programming fundamentals among players.

Objectives of the project

1. **Educational Integration:** To create a single-player Snake game that effectively demonstrates core Data Structures and Algorithms (DSA) concepts, allowing players to learn through gameplay.

2. **Enhanced User Experience:** To design an engaging and intuitive user interface that enhances player interaction and enjoyment.

3. **Efficient Resource Management:** To implement efficient data structures (e.g., linked lists, arrays) for managing game state, snake movement, and score tracking, optimizing performance and responsiveness.

4. **Algorithmic Understanding:** To incorporate and explain algorithms related to movement, collision detection, and scoring, providing players with insights into the underlying programming logic.

5. **Testing and Validation:** To develop comprehensive test cases that ensure the game functions correctly and reliably, enhancing the overall quality of the project.

6. **Future Expansion:** To lay the groundwork for potential future enhancements, such as multi-player modes and advanced graphics, promoting continuous learning and improvement.

Requirements of the system (Hardware, software)

Hardware Requirements

- **Processor:** Minimum 1 GHz or higher.
- **RAM:** At least 1 GB of memory.
- **Storage:** A few megabytes of available space for the game files.
- **Graphics:** Basic graphics capability to support text-based or simple 2D graphics.

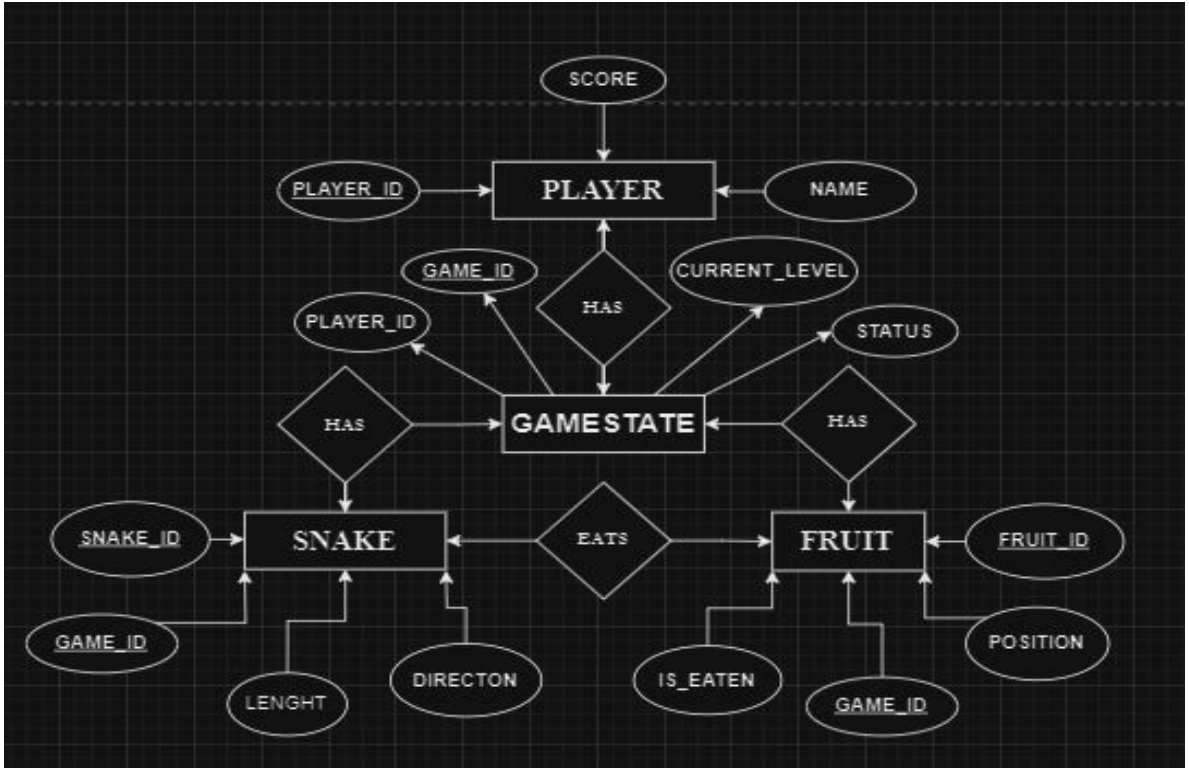
Software Requirements

- **Operating System:** Compatible with Windows, Linux, or macOS.
- **C Compiler:** GCC or any other C compiler to compile the source code.
- **Integrated Development Environment (IDE):** Code::Blocks, Dev-C++, or Visual Studio Code for code editing and debugging.

Additional Requirements

- **Input Device:** Keyboard for player controls.
- **Display:** A monitor with a minimum resolution of 800x600 for optimal gameplay visibility.

ER diagram of the proposed system



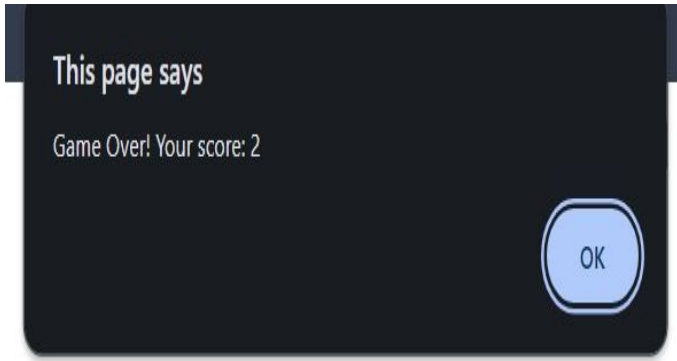
Front End

Snake Game

Score: 2

Restart Game

Snake Game



Score: 2

Data Structures And Concepts Used

1. Linked List (for the Snake):

- Represents the snake as a singly linked list, where each node is a segment.
- Each node stores the coordinates (x, y) and a pointer to the next segment.
- The head updates with new positions, while the tail moves forward when food is eaten.

2. Queue (for Movement):

- Maintains direction inputs (up, down, left, right) as the player presses keys.
- Ensures smooth processing of movements in the order received.

3. 2D Array (for Game Board):

- Represents the game board as a grid, with each cell indicating a position.
- Stores the snake's position and food locations, updating on each frame to reflect movement and consumption.

Algorithm

1. Initialize Game State

- Set up the game board as a 2D array.
- Create an initial snake represented as a linked list.
- Place the first piece of food at a random position on the board.
- Initialize the player's score to 0.

2. Main Game Loop

- Repeat Until Game Over:
  - **1. Handle User Input:**
    - Check for user key presses (up, down, left, right) and enqueue the direction.
  - **2. Process Movement:**
    - Dequeue the first direction from the queue to determine the snake's movement.
    - Update the snake's head position based on the direction.
  - **3. Check for Collisions:**
    - If the snake's head collides with the wall or itself, set game status to "Over."
    - If the snake's head collides with food:

Algorithm

- Increment the score.
- Add a new segment to the snake (create a new node in the linked list).
- Place new food at a random position on the board.

4. Update Snake Position:
- Move the segments of the snake accordingly.
- Update the tail's position if the snake has not eaten food.

5. Render Game Graphics:
- Clear the screen.
- Draw the game board.
- Render the snake and food based on their current positions.

6. Delay for Game Speed:
- Introduce a short delay to control the game speed.

3. End Game
- Display the final score and a game-over message.
- Prompt the user to play again or exit.

Time and Space Complexity

Time Complexity

1. **Initialization:**O(1).
2. **Main Game Loop:**
    - **User Input Handling:** O(1).
    - **Snake Movement:** O(1).
    - **Collision Detection: O**(1).
    - **Food Consumption:** O(1).
    - **Rendering Graphics:** O(m), where m is the number of cells to render (which is constant).

Overall, the time complexity for each frame in the main game loop is O(1), but when considering multiple frames, it can be viewed as O(k), where k is the number of frames until the game ends.

Space Complexity

Data Structures:

- **Linked List for Snake:** O(n), where n is the length of the snake.
- **Queue for Movements:** In the worst case, O(k), where k is the number of movement commands stored.
- **2D Array for Game Board:** O(1).

Overall, the space complexity of the program is dominated by the linked list for the snake, leading to a total space complexity of O(n + k), where n is the length of the snake and k is the number of movement commands in the queue.

Implementation

1. *initializeSnake(Snake snake):* Initializes the snake's starting position, length, and direction. It creates the head segment of the snake, sets it to the center of the board, and initializes the tail to the head.

2. *generateFood(Food food, Snake* snake): Randomly generates a position for food on the board. It ensures that the food does not appear on any part of the snake by checking the current segments of the snake against the generated position.

3. setup(); Sets up the SDL environment, creates a window and renderer, initializes the column and row dimensions based on window size, and seeds the random number generator. It also initializes the SnakeGame structure and calls `resetGame`.

4. *printBoard(Snake snake, Food* food): Displays the game board in the console. It draws walls, the snake, and the food at their respective positions. This function uses a console output to represent the game state.

5. *resetGame(SnakeGame game)*: Resets the game state by clearing the existing snake segments and reinitializing the score. It creates a new initial snake at the center of the board and calls `resetGoal` to set a new goal position.

6. *moveSnake(Snake snake, Food* food, int* gameOver): Moves the snake based on its current direction. It checks for keyboard input to change the direction of the snake and prevents the snake from reversing. It also checks for collisions with walls or itself, updating the game-over state as necessary.

7. *resetGoal(SnakeGame game)*: Resets the goal position by randomly generating a new position within the grid boundaries.

Implementation

8. *gameOverScreen(SnakeGame game)*:Displays a game-over screen. In this implementation, it sets up the rendering context for a potential game-over message and resets the game state.

9. *checkGoal(SnakeGame game, int column, int row):**Checks if the current position matches the goal position. This function is used to determine if the snake has "eaten" the food.

10. *submitMove(SnakeGame game, char input)*:Processes directional input from the user. It updates the snake's direction based on the key pressed, allowing for movement in the four cardinal directions, and handles a special case for reversing the snake.

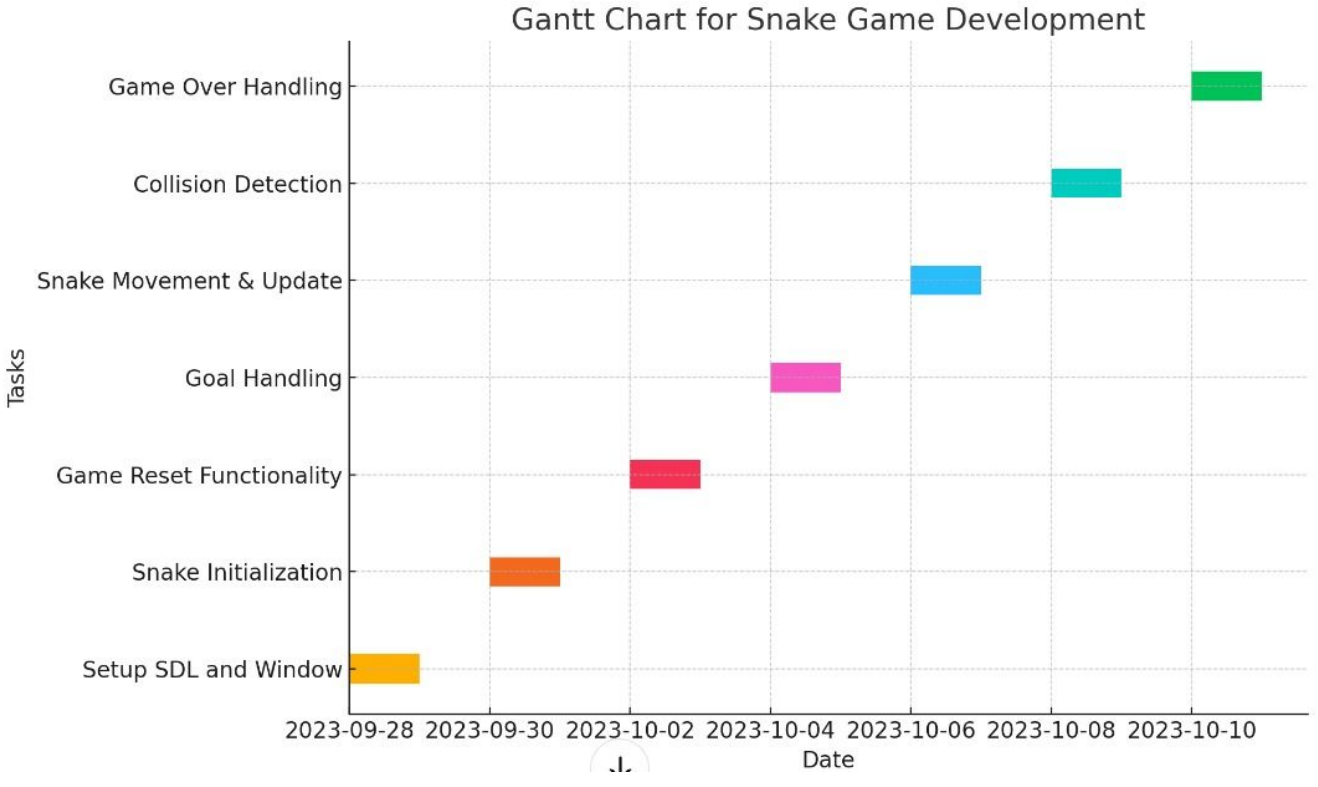
11. *update(SnakeGame game)*:Updates the snake's position by creating a new head node based on the current direction. It checks for wall collisions and food consumption. If the snake eats food, the score is incremented, and the goal is reset; if not, it removes the tail of the snake.

12. *display(SnakeGame game):**Renders the game on the SDL window. It clears the renderer, draws the snake and food on the screen, and checks for collisions with itself.

13. *checkCollision(SnakeGame game)*:Checks if the snake collides with itself by comparing the position of the head with all other segments. If a collision is detected, it returns true.

14. main(): The entry point of the program. It initializes the game, enters the main game loop to handle input, updates the game state, and renders the graphics. It also handles quitting the game and cleans up resources at the end.

Gantt Chart


Gantt Chart for Snake Game Development
Game Over Handling
Collision Detection
Snake Movement & Update
Goal Handling
Game Reset Functionality
Snake Initialization
Setup SDL and Window
Tasks
2023-09-28
2023-09-30
2023-10-02
2023-10-04
2023-10-06
2023-10-08
2023-10-10
Date

Test Cases

1. **Basic Movement**
   - **Input:** Use 'w', 'a', 's', 'd' to move the snake.
   - **Expected Output:** The snake should move in the specified direction.
2. **Food Generation**
   - **Input:** Start the game and check for food generation.
   - **Expected Output:** Food should appear randomly on the board, not overlapping the snake.
3. **Eating Food**
   - **Input:** Move the snake to the food's position.
   - **Expected Output:** The snake's length should increase and the score should increment.
4. **Collision with Walls**
   - **Input:** Move the snake into the wall boundaries.
   - **Expected Output:** The game should end with a "Game Over" state.
5. **Self-Collision**
   - **Input:** Move the snake in a way that it collides with itself.
   - **Expected Output:** The game should end with a "Game Over" state.
6. **Restart Game**
   - **Input:** Press the spacebar after a game over.
   - **Expected Output:** The game should reset and allow for new play.

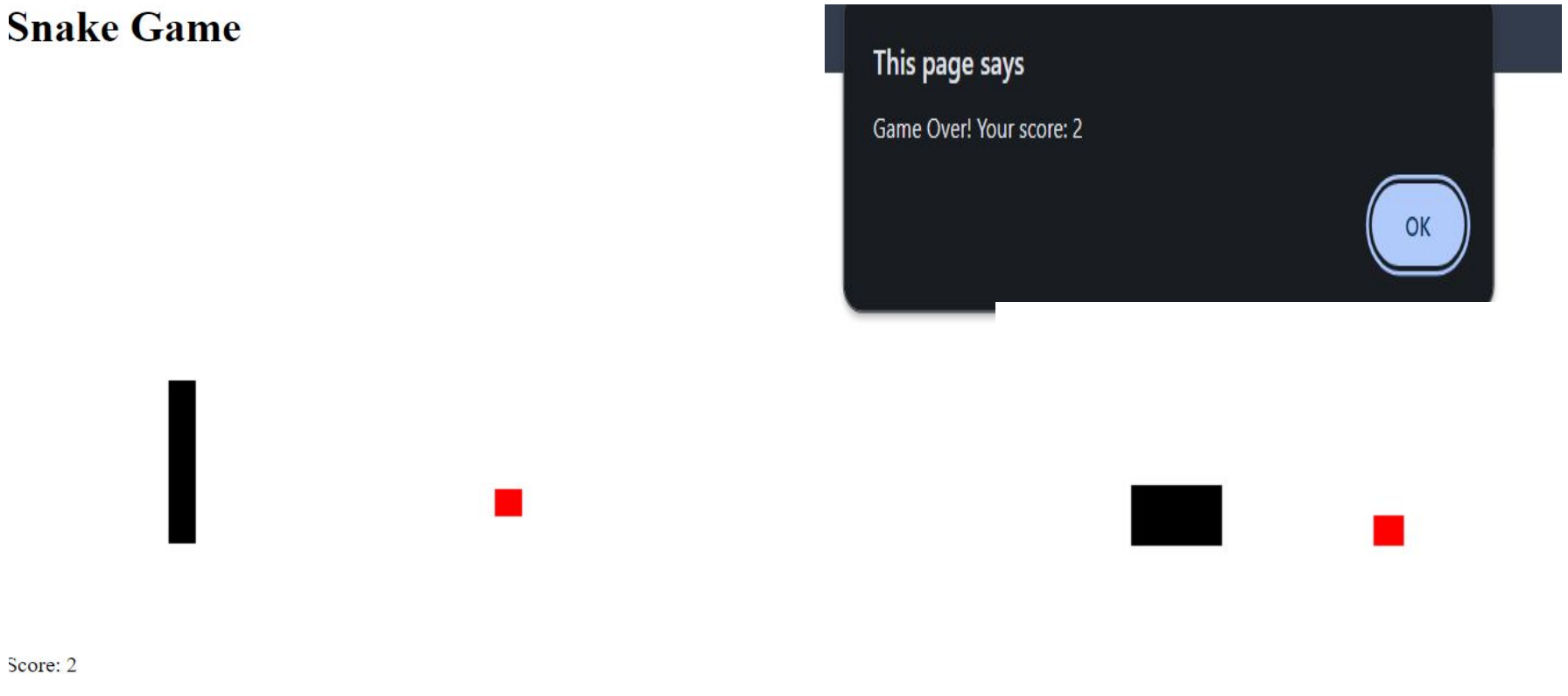
Challenges and Solution

1. **Handling Game Over State**
- **Challenge:** Ensure that the game properly transitions to the game over state and allows for restarts.
- **Solution:** Implement a clear game state check and reset logic in the main game loop.
2. **Rendering Performance**
- **Challenge:** The rendering may become slow with a longer snake due to constant memory allocation and deallocation.
- **Solution:** Consider implementing a fixed-size buffer for the snake's body or using a more efficient data structure (like a circular buffer).
3. **Input Responsiveness**
- **Challenge:** Input handling may lag, especially with continuous key presses.
- **Solution:** Use event polling instead of blocking input calls, ensuring smooth input handling during the game loop.
4. **Memory Management**
- **Challenge:** Properly manage memory allocation to avoid memory leaks when segments are added/removed.
- **Solution:** Ensure every allocated segment is freed when it's no longer needed and check for potential leaks.
5. **Random Food Generation**
- **Challenge:** Ensure food generation does not overlap with the snake.
- **Solution:** Implement a robust checking mechanism before placing food.

Future Scope

1. **Graphical Enhancements**
   - **Add graphics**: Replace the ASCII representation of the snake and food with graphics, improving visual appeal.
   - **Animations**: Implement animations for movement and food generation.
2. **Sound Effects**
   - Incorporate sound effects for eating food and game over events to enhance user experience.
3. **Scoreboard System**
   - Create a scoring system with high scores saved to a file to track player performance over time.
4. **Difficulty Levels**
   - Introduce multiple difficulty levels that affect snake speed and game complexity.
5. **Multiplayer Mode**
   - Add a feature for two players to compete on the same screen, increasing complexity and competitiveness.
6. **Obstacle Implementation**
   - Introduce static or moving obstacles on the board to increase the game's difficulty.
7. **Cross-Platform Compatibility**
   - Ensure the game can be compiled and run on different operating systems with minimal changes.

Output SS



Conclusion

The Snake game implemented in C using SDL2 demonstrates foundational principles of game development, including graphics rendering, user input handling, and game state management. Through this project, we explored essential programming concepts such as data structures, memory management, and real-time event handling.

The game successfully incorporates core mechanics like movement, food generation, scoring, and collision detection, offering an engaging experience reminiscent of classic arcade games. We identified and addressed several challenges, such as managing game state transitions and ensuring efficient rendering, laying the groundwork for further enhancements.

Looking ahead, there is substantial scope for improvement and expansion. Adding graphical assets, sound effects, and advanced features like multiplayer mode or difficulty levels can enhance user engagement and broaden the game's appeal. By continuing to build upon this project, we can create a more polished and enjoyable gaming experience while solidifying our understanding of software development principles.

This project not only serves as a practical exercise in programming but also opens the door to further exploration in game design, paving the way for future projects in this dynamic field.

References

IEEE REFERENCE PAPERS:

1. **Snake Game AI: Movement Rating Functions and Evolutionary Algorithm-based Optimization**
   J. -F. Yeh, P. -H. Su, S. -H. Huang and T. -C. Chiang, "Snake game AI: Movement rating functions and evolutionary algorithm-based optimization," 2016 Conference on Technologies and Applications of Artificial Intelligence (TAAI), Hsinchu, Taiwan, 2016, pp. 256-261, doi: 10.1109/TAAI.2016.7880166. keywords: {Games;Artificial intelligence;Evolutionary computation;Optimization;Computers;Turning;Weapons;game;artificial intelligence (AI);snake;evolutionary algorithm},

2. **Playing the Game of Snake with Limited Knowledge: Unsupervised Neuro-Controllers Trained Using Particle Swarm Optimization**
   C. J. van Rooyen, W. S. van Heerden and C. W. Cleghorn, "Playing the game of snake with limited knowledge: Unsupervised neuro-controllers trained using particle swarm optimization," 2017 IEEE 4th International Conference on Soft Computing & Machine Intelligence (ISCMI), Mauritius, 2017, pp. 80-84, doi: 10.1109/ISCMI.2017.8279602. keywords: {Games;Neurons;Artificial intelligence;Particle swarm optimization;Computer science;Electronic mail;Training;Artificial intelligence;neuro-controllers;particle swarm optimization;agents;game;Snake},